

Lab 2: Agile Backlog Creation & Sprint Simulation in Jira

OpenAPI Mock Server Generator

Name: Bharath CD

SRN: PES1UG24CS562

Section: 5J

1. Epics

The following five Epics were created in Jira, derived from the OpenAPI Mock Server Generator SRS (FR-001–FR-005, NFR-001–NFR-002) and its UML use-case diagram (actors: API Developer, QA Engineer):

Epic 1: OpenAPI Specification Upload & Validation

Enable an API Developer to upload a JSON/YAML OpenAPI specification, have it validated by the system, and see clear validation errors when invalid. (FR-001, FR-002)

Epic 2: Mock Server Generation

Enable the system to generate a mock server containing every endpoint defined in a valid OpenAPI specification, and let the API Developer view the generated endpoint list. (FR-003)

Epic 3: Mock Server Lifecycle Control

Enable the API Developer to start, stop, and restart the generated mock server, with the system reflecting current server status. (FR-004)

Epic 4: API Endpoint Testing

Enable a QA Engineer to send test requests to generated endpoints, view mock responses, and retry after a failed request. (FR-005)

Epic 5: System Performance & Reliability

Ensure the system meets performance targets for spec validation and mock server generation under load. (NFR-001, NFR-002)

Backlog View — Epics and User Stories (before estimation)

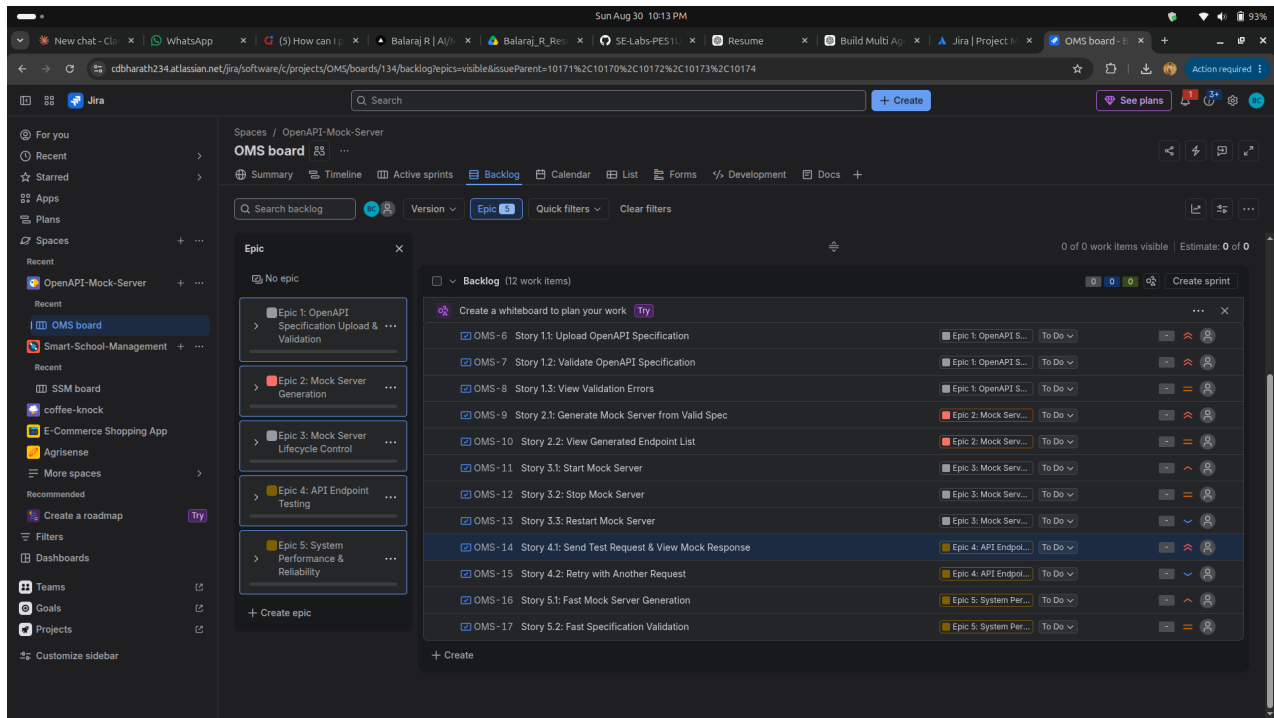


Figure 1: Full backlog showing all 5 Epics and their 12 linked User Stories, each in To Do status, with priority flags assigned per the SRS's Priority column.

2. Story Point Estimation

Story points were assigned using the Fibonacci scale, reflecting the relative complexity, effort, and uncertainty of each story — e.g. Story 2.1: Generate Mock Server (8 points, the system's core function) and Story 5.1: Fast Mock Server Generation (8 points, high uncertainty performance work).

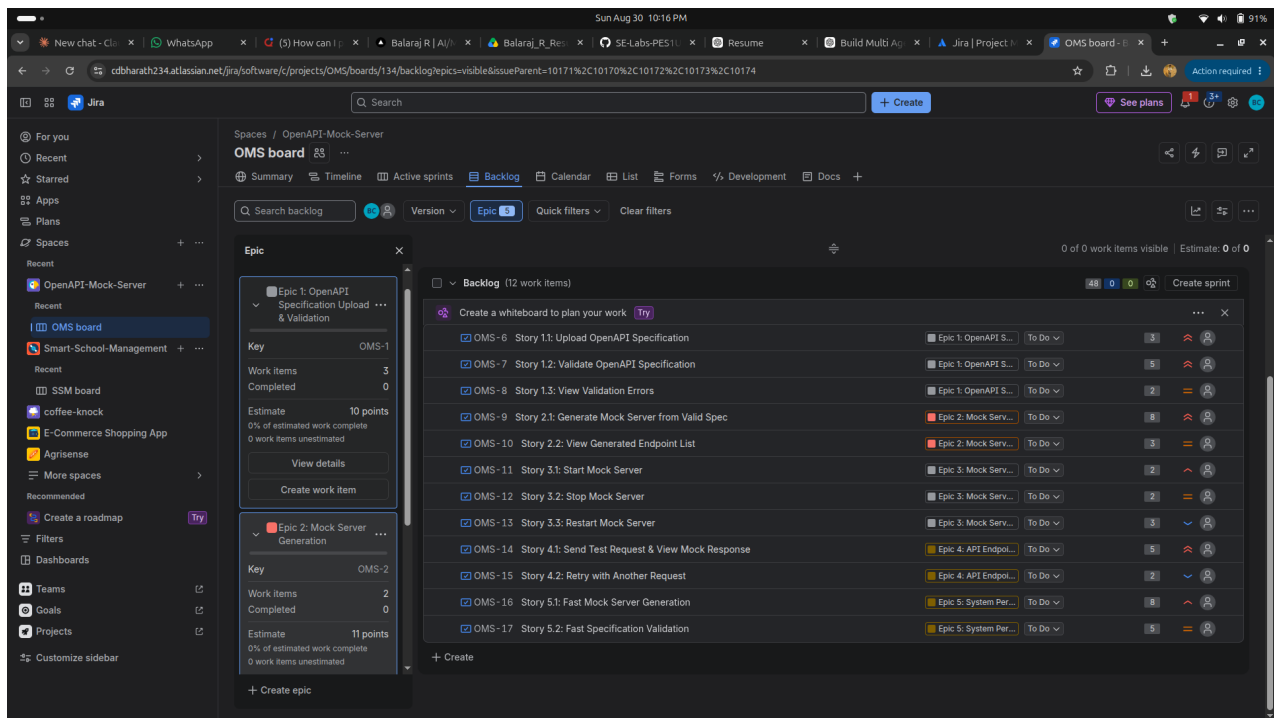


Figure 2: Backlog with Story Points assigned to each item and the running Epic estimate totals (e.g. Epic 1: 10 points across 3 stories, Epic 2: 11 points across 2 stories). The full backlog sums to 48 total story points across all 12 stories.

3. Sprint Execution

Sprint 1 (26 points: Stories 1.1, 1.2, 2.1, 2.2, 3.1, 4.1) was run and completed first. No direct screenshot of Sprint 1's active board was captured, but its outcome is confirmed by the Epic completion bars visible in Figure 3 below: Epic 2 (Mock Server Generation) reached 100% complete, Epic 1 reached roughly two-thirds complete (2 of 3 stories — the third, Story 1.3, was never part of Sprint 1), and Epic 3 and Epic 4 show partial progress from their one Sprint-1 story each (Story 3.1 and Story 4.1). All 6 of Sprint 1's committed stories reached Done, a full 26 of 26 points completed with no rollover.

Sprint 2 Created with Remaining Backlog Items

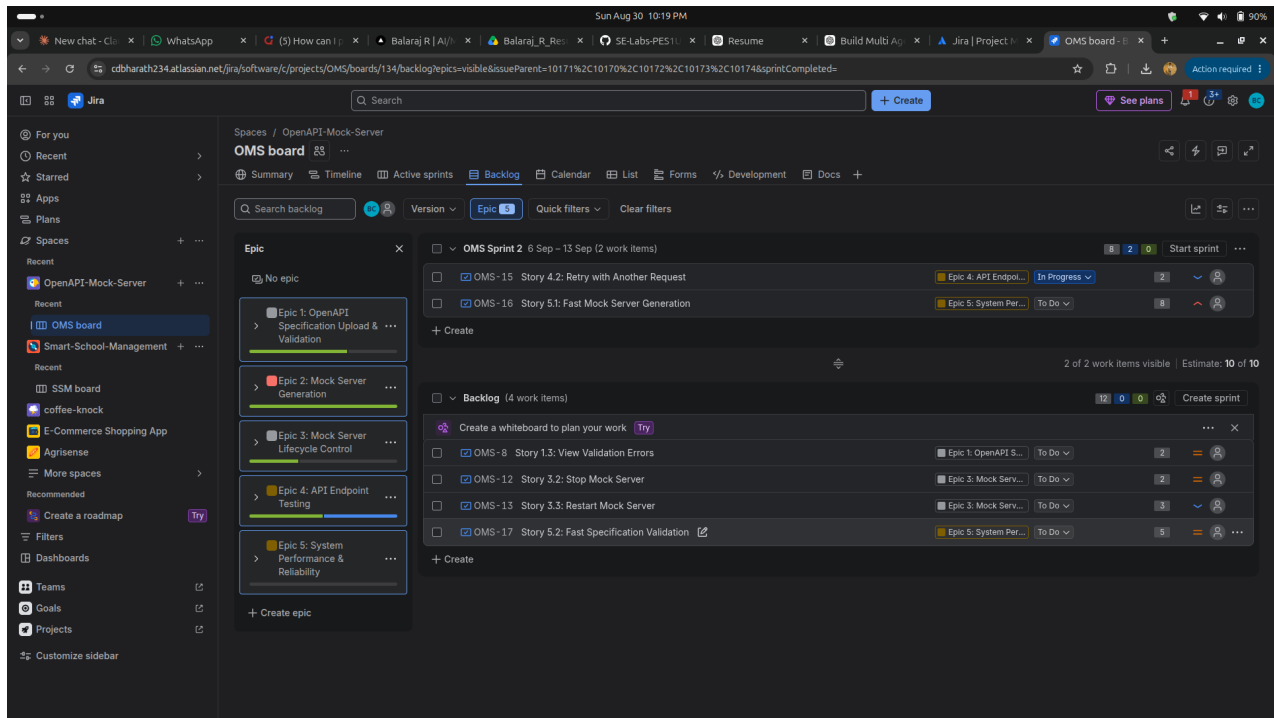


Figure 3: With Sprint 1 fully complete, Sprint 2 was created and populated with 2 stories initially (Story 4.2 and Story 5.1, In Progress / To Do), while the remaining 4 unstarted stories (1.3, 3.2, 3.3, 5.2) sat in the general backlog. Epic progress bars confirm Epic 2 at 100% and partial completion for Epics 1, 3, and 4.

Sprint 2 — Active Sprint Board

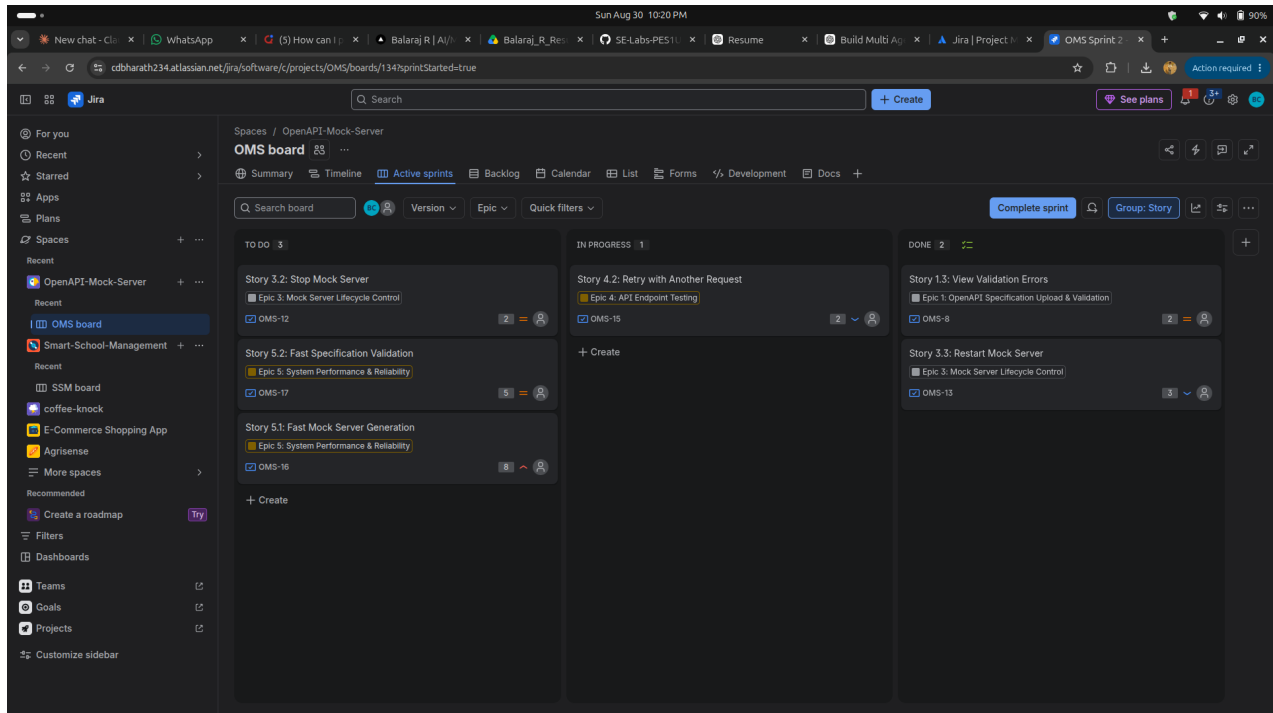


Figure 4: Sprint 2 fully populated with all 6 remaining stories (22 points total). At the time of this capture: To Do — Story 3.2 (2 pts), Story 5.2 (5 pts), Story 5.1 (8 pts); In Progress — Story 4.2 (2 pts); Done — Story 1.3 (2 pts), Story 3.3 (3 pts). That's 5 of 22 points completed at this stage of Sprint 2.

Note: Sprint 2 was not yet fully completed ("Complete sprint" not yet clicked) at the time this screenshot was taken — 3 stories were still in To Do. Make sure Sprint 2 is driven to 100% Done and completed before final submission, since the lab requires finishing 2 full sprints.

4. Burndown Chart

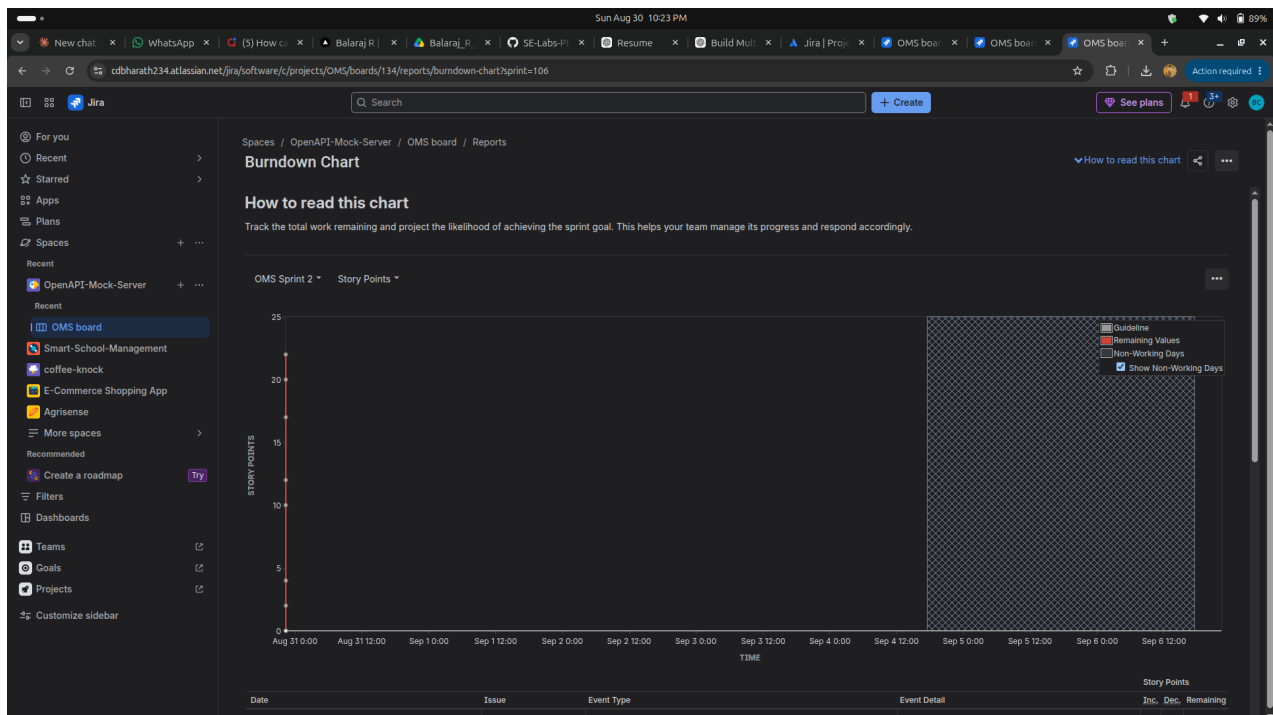


Figure 5: Burndown Chart for OMS Sprint 2 (Story Points metric). The chart currently shows only a starting data point at the sprint's beginning, with most of the date range shaded as non-working days, and no visible declining "Remaining Values" line yet.

Note: this doesn't yet visually reflect the progress already shown on the Sprint 2 board in Figure 4 (5 of 22 points done, i.e. some decline expected). This is likely because the chart was captured very shortly after Sprint 2 started, before enough time had passed for the burndown to register, or because the sprint's working-day calendar needs checking. It's worth re-visiting Reports → Burndown Chart after Sprint 2 is fully completed and re-capturing this screenshot, since a flat/undeclined line here won't clearly demonstrate progress to your instructor.

5. Reflection Questions

1. Did your estimations reflect the actual effort?

For the most part, yes. Sprint 1's highest-point story, Story 2.1: Generate Mock Server (8 points, flagged as the system's core and most complex function), was completed successfully alongside 5 other stories, all 26 points landing in Done with no rollover — suggesting the 8-point estimate wasn't an obstacle when the story was prioritized early. In Sprint 2, the two remaining high-point stories, Story 5.1: Fast Mock Server Generation (8 points) and Story 5.2: Fast Specification Validation (5 points), were still sitting untouched in To Do while smaller stories like Story 1.3 (2 points) and Story 3.3 (3 points) were already Done. This matches the original reasoning that the two NFR/performance stories carried higher uncertainty — they appear to be the ones the team is leaving until later in the sprint, which is a reasonable real-world pattern for less-understood, performance-tuning work.

2. Was your backlog well-prioritized?

Yes, broadly. The four Highest-priority stories from the SRS (1.1, 1.2, 2.1, 4.1) were exactly the ones pulled into Sprint 1 alongside Story 3.1 (High priority) and Story 2.2 (Medium priority), and all six reached Done. That matches a sensible Highest → High → Medium ordering. In Sprint 2, Story 3.2 (Medium) and Story 5.2 (Medium) remain in To Do while Story 4.2 (Low priority) is already In Progress — a minor inversion worth noting, since a Low-priority item is currently ahead of two Medium-priority ones in the flow.

3. How did your simulated sprint align with your plan?

Sprint 1 aligned very well with the plan: all 6 committed stories (26 points) reached Done with zero rollover, a full 100% completion rate. Sprint 2, still in progress at the time of the board screenshot, had completed 2 of 6 stories (5 of 22 points, about 23%), with one story In Progress and three still in To Do. This two-sprint picture shows an initial sprint that matched capacity well, followed by a second sprint that was still being worked through when captured — it should be driven to completion (all columns moved to Done, then "Complete sprint" clicked) before this is submitted as final evidence.

4. What insights did the burndown chart give about your team's capacity?

Sprint 1 completing all 26 committed points with no leftover work suggests the team's realistic capacity is at least around 26 points per sprint-equivalent, and Sprint 2's 22-point commitment sits comfortably within that range rather than over-committing. That said, the captured Burndown Chart itself doesn't yet show a useful capacity signal — it displays only a starting point with the rest of the date range shaded as non-working days, which doesn't line up with the 5 points already completed on the Sprint 2 board. Burndown charts are most informative once a sprint is fully closed out, so re-capturing this chart after completing Sprint 2 would give a clearer picture of whether the team is tracking ahead of, on, or behind the ideal guideline.

End of report. Deliverables included: Epics (Section 1), Story Points (Section 2), Sprint boards (Section 3), Burndown Chart (Section 4), and Reflection Questions (Section 5).